CREATE-ALT/GEN-TLE · SERVER/ · B_INITIAL-API E0AFD80

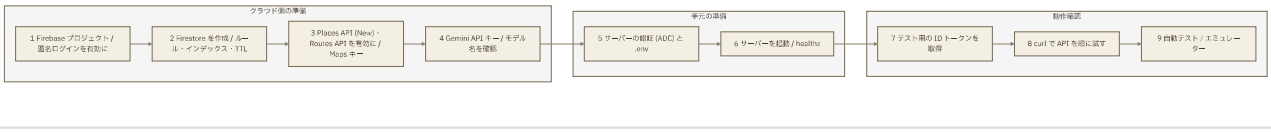
gen-TLE 動作確認ガイド

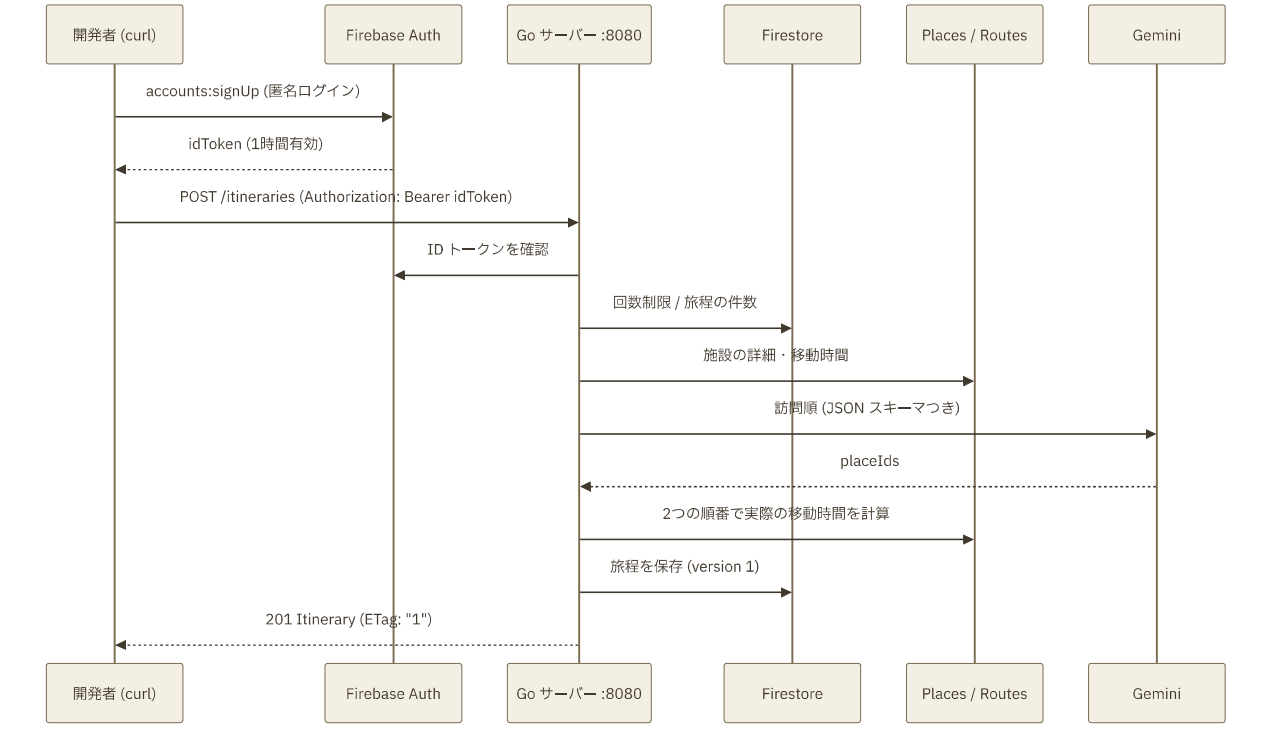
Go の旅程サーバーを、本物の Firebase・Firestore・Google Maps・Gemini につないで動かし、16 本の API を curl で順に試すまでの手順です。上から順に進めれば、旅程の作成から遅延時の組み直し、削除まで一通り確認できます。

- 所要時間の目安は 30〜60 分です。Firestore のインデックス作成待ちに数分かかります。
- Google Maps は課金の設定が必要です。1 回通して試すと、Places と Routes を数十回、Gemini を 10 回ほど呼びます。
- API キーやサービスアカウントの JSON は、Git に追加しないでください（`.env` は `.gitignore` 済み）。

全体の流れ

準備から動作確認まで





手順 0

必要なもの

もの	用途	確認
Go 1.26 以上	サーバーの起動とテスト	`go version`
Google Cloud プロジェクト（課金を有効に）	Firebase・Firestore・Maps をまとめて使う。Firebase プロジェクトと同じものにする	Cloud Console の課金
gcloud CLI	API の有効化と、サーバーの認証（ADC）	`gcloud --version`
Firebase CLI	Firestore のルール・インデックスの反映、エミュレーター	`npm install -g firebase-tools` のあと `firebase --version`
Google AI Studio のアカウント	Gemini API キーの発行	aistudio.google.com
curl と jq	API を試す	`jq --version`

1 Firebase プロジェクトと匿名ログイン

1. Firebase コンソールでプロジェクトを作ります（既存の Google Cloud プロジェクトを選べます）。
2. 「Authentication」→「Sign-in method」で「匿名」を有効にします。アプリは起動時に匿名ログインし、その ID トークンをサーバーに送ります。
3. 「プロジェクトの設定」→「全般」で、**プロジェクト ID** と **ウェブ API キー** を控えます。ウェブ API キーは、手順 7 でテスト用の ID トークンを取るときだけ使います。

2 Firestore の作成とルール・インデックス・TTL

1. Firebase コンソールの「Firestore Database」で、データベース（ネイティブモード、例：asia-northeast1）を作ります。
2. `server/` に `firebase.json` を作り、リポジトリにあるルール・インデックスの設定を反映します。インデックスには、④ 一覧に使う複合インデックスと、3 つのコレクションの TTL（`expiresAt`）の設定が入っています。

server/firebase.json（新規作成）

```
{
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  }
}
```

server/ で実行

```
firebase login
firebase deploy --only firestore:rules,firestore:indexes --project YOUR_PROJECT_ID
```

確認 Firebase コンソールの「Firestore」→「インデックス」で、itineraries の複合インデックス（ownerUid ↑ / date ↓ / __name__ ↓）が「有効」になるまで待ちます。作成中に ④ を呼ぶと失敗します（下のトラブルシューティング参照）。

ルールはアプリからの直接アクセスをすべて拒否します。サーバーは Admin SDK を使うので、ルールの影響を受けません（代わりに手順 5 の IAM 権限が必要です）。

3 Google Maps の API と API キー

API を有効にする

```
gcloud services enable places.googleapis.com routes.googleapis.com --project YOUR_PROJE
```

1. 認証情報で「API キーを作成」します。
2. キーの「API の制限」で、**Places API (New)** と **Routes API** だけを許可します。サーバーから使うキーなので、HTTP リファラーの制限は付けません。
3. このキーを、手順 5 の GOOGLE_MAPS_API_KEY に入れます。アプリ（google_maps_flutter の地図表示）用のキーとは分けてください。

キーの確認（Places API (New) の Text Search）

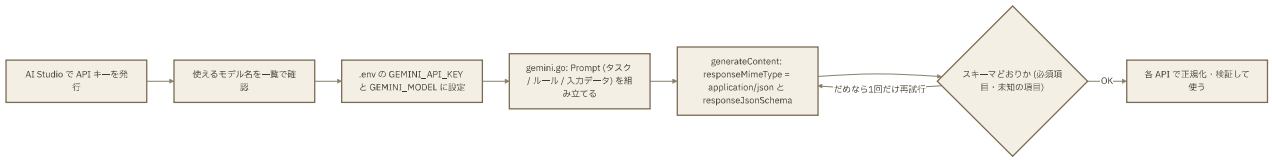
```
curl -sS -X POST https://places.googleapis.com/v1/places:searchText \
  -H "Content-Type: application/json" \
  -H "X-Goog-Api-Key: $GOOGLE_MAPS_API_KEY" \
  -H "X-Goog-FieldMask: places.id,places.displayName" \
  -d '{"textQuery": "東京駅", "languageCode": "ja"}' | jq .
```

確認 places[0].id（ChIJ で始まる ID）が返れば OK です。403 なら、API が有効になっていないか、

キーの制限が合っていません。

4 Gemini API の準備

サーバーは Gemini を REST（`generateContent`）で呼び、出力を JSON スキーマで縛ります。キーとモデル名を用意して、サーバーと同じ形の呼び出しが通るか先に確かめます。



1. Google AI Studio の API キーのページで、手順 1 のプロジェクトを選んでキーを作ります。
2. 使えるモデルを一覧で確認し、名前（`models/` を除いた部分）を `GEMINI_MODEL` にします。構造化出力（JSON スキーマ）に対応したモデルを選んでください。

使えるモデルの一覧

```
curl -sS https://generativelanguage.googleapis.com/v1beta/models \
  -H "x-goog-api-key: $GEMINI_API_KEY" \
  | jq -r '.models[] | select(.supportedGenerationMethods | index("generateContent")) | .name'
```

サーバーと同じ形で呼べるか（JSON スキーマつき）

```
curl -sS "https://generativelanguage.googleapis.com/v1beta/models/$GEMINI_MODEL:generateContent" \
  -H "Content-Type: application/json" \
  -H "x-goog-api-key: $GEMINI_API_KEY" \
  -d '{
    "contents": [{"role": "user", "parts": [{"text": "海が見えるカフェに行きたい、という希望を施設検索の
    "generationConfig": {
      "responseMimeType": "application/json",
      "temperature": 0.2,
      "responseJsonSchema": {
        "type": "object",
        "properties": {
          "queries": {"type": "array", "maxItems": 5, "items": {
            "type": "object",
            "properties": {"query": {"type": "string"}, "intent": {"type": "string"}},
            "required": ["query", "intent"]}}
        },
        "required": ["queries"]
      }
    }
  }' | jq -r '.candidates[0].content.parts[0].text' | jq .
```

確認 `{"queries": [{"query": "...", "intent": "..."}]}` の形の JSON だけが返れば OK です。② `POST /places/interpret` の 1 回目の呼び出しと同じ形です。

Gemini を呼ぶ API ② 2 回、③⑥ 各 1 回、⑦ 1 回（時刻調整だけで済むときは 0 回）、⑧ 案ごとに 1 回、⑩ 1 回です。応答がスキーマに合わないときは、もう 1 回だけ呼びます。無料枠では 1 分あたりの回数制限に当たりやすいので、429 が出たら少し待ってください。

5 サーバーの認証と環境変数

Firebase Admin SDK と Firestore は Application Default Credentials（ADC）で認証します。手元で試すなら、自分の Google アカウントで ADC を作るのが簡単です。

自分のアカウントで ADC を作る

```
gcloud auth application-default login
gcloud auth application-default set-quota-project YOUR_PROJECT_ID
```

コピー

サービスアカウントを使う場合は、Firestore を読み書きできるロール（Cloud Datastore ユーザー）を付けて JSON キーを作り、その絶対パスを `GOOGLE_APPLICATION_CREDENTIALS` に入れます。自分のアカウントの ADC を使うなら、その行は消してください。

server/ で実行

```
cp .env.example .env
```

コピー

server/.env（値を書き換える）

```
FIREBASE_PROJECT_ID=your-firebase-project-id
GOOGLE_MAPS_API_KEY=手順3のキー
GEMINI_API_KEY=手順4のキー
GEMINI_MODEL=手順4で選んだモデル名
PORT=8080
```

コピー

6 サーバーの起動

server/ で実行（.env は自動では読まないので、シェルに読み込む）

```
set -a
. ./.env
set +a
go run ./cmd/server
```

コピー

```
go run ./cmd/server
```

別のターミナルで

```
curl -sS http://localhost:8080/healthz
```

コピー

確認 `{"status":"ok"}` が返れば起動しています。起動時に「～を設定してください」と出て止まる場合は、`.env` の読み込みができていません。

7 テスト用の ID トークンを取る

アプリの代わりに、Firebase Auth の REST API で匿名ユーザーを作り、ID トークンを取ります。トークンは 1 時間で切れるので、401 が出たらもう一度取ってください。

ターミナルで（以降の手順はすべて同じターミナルで）

```
export BASE=http://localhost:8080
export WEB_API_KEY=手順1のウェブAPIキー
export TOKEN=$(curl -sS -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=$W
  -H "Content-Type: application/json" -d '{"returnSecureToken": true}' | jq -r .idToken)

# 以降の呼び出しを短くする
api() { curl -sS -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" "$@"; }
echo "${TOKEN:0:20}..."
```

コピー

同じトークンを使い続けると、同じ匿名ユーザー（uid）として扱われます。新しく `signUp` すると別のユーザーになり、前の旅程は 404 になります。

8 API を順に試す

図のとおり、前の API の出力（placeId、旅程の id、version など）を次の API に渡しながら進めます。旅程を変える API は、毎回最新の version を `If-Match` に入れます。



8-1 施設を探す（① ② ⑫ ⑬）

① 入力補完と ② 自由文の解釈

```
api -G "$BASE/places/autocomplete" --data-urlencode "q=東京駅" | jq .
api -X POST "$BASE/places/interpret" -d '{"text": "静かな寺と、海が見えるカフェに行きたい", "near": {"l
```

コピー

⑫ 地図の近くの施設と ⑬ 施設の詳細（その日の営業時間）

```
api -G "$BASE/places/nearby" --data-urlencode "lat=35.681" --data-urlencode "lng=139.76
export START=$(api -G "$BASE/places/autocomplete" --data-urlencode "q=東京駅" | jq -r '.candidate
export P1=$(api -G "$BASE/places/autocomplete" --data-urlencode "q=皇居東御苑" | jq -r '.candidate
export P2=$(api -G "$BASE/places/autocomplete" --data-urlencode "q=三菱一号館美術館" | jq -r '.cand
api -G "$BASE/places/$P2" --data-urlencode "date=$(date +%F)" | jq .
```

コピー

8-2 旅程を作る（③）

旅行日（DATE）は、今日から 100 日以内にしてください。それより先は、公共交通の経路が検索できず徒歩だけになります。

③ 旅程の作成（結果を `trip.json` に保存）

```
export DATE=$(date -d "+3 days" +%F 2>/dev/null || date -v+3d +%F)
jq -n --arg d "$DATE" --arg s "$START" --arg p1 "$P1" --arg p2 "$P2" '{
  title: "東京駅まわり 日帰り", date: $d,
  start: {placeId: $s, departAt: "09:00"},
  end:   {placeId: $s, arriveBy: "19:00"},
  places: [{placeId: $p1, priority: "must", stayMin: 60}, {placeId: $p2, priority: "want", stayM
}' | api -X POST "$BASE/itineraries" -d @- | tee trip.json | jq '{id, version, items: [.items[]
export TRIP=$(jq -r .id trip.json)
```

コピー

確認 201 で `legs` の最後が `"toItemId": "end"`、`summary.endAt` と `end.arriveAt` が同じ時刻になっていれば OK です。地図上の地点を出発地にするなら、`start` を `{"location": {"lat": 35.68, "lng": 139.76}, "name": "丸の内広場", "departAt": "09:00"}` にします。

8-3 一覧と取得（④ ⑤）

④ 一覧と ⑤ 取得（ETag に version が入る）

```
api -G "$BASE/me/itineraries" --data-urlencode "limit=5" | jq .
api -i "$BASE/itineraries/$TRIP" | grep -i '^etag'
ver() { api "$BASE/itineraries/$TRIP" | jq .version; }
```

コピー

8-4 空き時間の提案と追加（⑩ ⑪）

最後の予定のあと～18:00 を空き時間として提案を受けます（終わる場所に着きたい時刻を 19:00 にしているため）。最後の予定が 18:00 を過ぎる場合は、TO を後ろにずらしてください。

⑩ 提案（suggestions と ideas）

```
api "$BASE/itineraries/$TRIP" > trip.json
```

コピー

```
export LAST=$(jq -r '.items[-1].itemId' trip.json)
export FROM=$(jq -r '.items[-1].endAt' trip.json)
export TO="${DATE}T18:00:00+09:00"
api -G "$BASE/itineraries/$TRIP/suggestions" --data-urlencode "from=$FROM" --data-urlencode "to=
```

⑪ 1件目の提案を追加

コピー

```
jq '.suggestions[0] | {placeId: .place.placeId, stayMin, priority: "optional", insertAf
  | api -X POST "$BASE/itineraries/$TRIP/items" -H "If-Match: $(ver)" -d @- | jq '{version, item
```

8-5 編集（⑥）

⑥ 行き先を全件送り直し、終わる時刻を変える

コピー

```
api "$BASE/itineraries/$TRIP" \
  | jq '{places: [.items[] | {placeId, priority, stayMin}], end: {placeId: .end.placeId, arriveB
  | api -X PUT "$BASE/itineraries/$TRIP/places" -H "If-Match: $(ver)" -d @- | jq '{version, end,
```

8-6 遅延時の組み直し（⑦ ⑧ ⑨）

⑦ 1つ目の予定で 60 分遅れた

コピー

```
export CUR=$(api "$BASE/itineraries/$TRIP" | jq -r '.items[0].itemId')
api -X POST "$BASE/itineraries/$TRIP/replans" -d "{\"currentItemId\": \"$CUR\", \"delayMin\": 60
  | tee replan.json | jq '{needsChange, warnings, options: [.options[] | {optionId, source, labe
export RP=$(jq -r .replanId replan.json)
export OP=$(jq -r '.options[0].optionId' replan.json)
```

⑧ 理由文（pending の間は 1 秒ごとに取り直す）

コピー

```
for i in 1 2 3 4 5 6 7 8 9 10; do
  R=$(api -G "$BASE/itineraries/$TRIP/replans/$RP/explanation" --data-urlencode "optionId=$OP")
  echo "$R" | jq -c .
  [ "$(echo "$R" | jq -r .status)" != "pending" ] && break
  sleep 1
done
```

⑨ 案を確定（If-Match は basedOnVersion）

コピー

```
api -X POST "$BASE/itineraries/$TRIP/replans/$RP/accept" -H "If-Match: $(jq .basedOnVersion repla
  -d "{\"optionId\": \"$OP\"}" | jq '{version, items: [.items[] | {name, status, endAt}]}'
```

確認 ⑦ で [illegible] なら、遅れが待ち時間に収まっています。[illegible] を増やして試して

確認 ⑦ で needsChange: false なら、遅れが待ち時間に収まっています。delayMin を増やして試してください。⑨ の後は 1 つ目の予定が current、それより前が done になります。15 分を過ぎた案を確定すると 410 です。

8-7 名前の変更・コピー・削除（⑭ ⑯ ⑮）

⑭ ⑯ ⑮

```
api -X PATCH "$BASE/itineraries/$TRIP" -H "If-Match: $(ver)" -d '{"title": "東京駅まわり
export COPY=$(api -X POST "$BASE/itineraries/$TRIP/copy" -d "{\"date\": \"$(date -d "+10 days" +
api -X DELETE "$BASE/itineraries/$TRIP" -H "If-Match: $(ver)" -o /dev/null -w "%{http_code}\n"
api -X DELETE "$BASE/itineraries/$COPY" -H "If-Match: 1" -o /dev/null -w "%{http_code}\n"
```

コピー

確認 削除は 204 を返し、その後の ⑤ は 404 になります。

9 自動テストとエミュレーター

通常のテストは、認証・保存先・Maps・Gemini の応答を差し替えて動くので、課金はかかりません。Firestore のトランザクション（同時更新、組み直し案、削除）は、エミュレーターで確かめます。

server/ で実行

```
go test ./...
go test -race ./...
go vet ./...
```

コピー

Firestore エミュレーターでのテスト（Java が必要）

```
firebase emulators:exec --only firestore --project demo-iterinerary \
  "go test ./internal/service -run TestFirestoreTransactions -count=1 -v"
```

コピー

emulators:exec が FIRESTORE_EMULATOR_HOST を設定し、テストはエミュレーターだけにつなぎます。この環境変数がないと、このテストはスキップされます。開発中のクラウド環境では実行できなかったので、手元で一度実行してください。

困ったとき

トラブルシューティング

症状	よくある原因	対処
起動時に「～を設定してください」	.env がシェルに読み込まれて	手順 6 の set -a ～ set

起動時に「キーを設定してください」	.env がシェルに読み込まれていない	手順 5 の `set -a; . ./.env; set +a` を同じシェルで実行する
起動時や保存時に権限エラー	ADC がない、クォータプロジェクトが違う、サービスアカウントに Firestore の権限がない	手順 5 をやり直す。サービスアカウントには Cloud Datastore ユーザーのロールを付ける
401 UNAUTHENTICATED	トークンの期限切れ（1 時間）、別プロジェクトのトークン、匿名ログインが無効	手順 7 でトークンを取り直す。`FIREBASE_PROJECT_ID` とウェブ API キーが同じプロジェクトか確かめる
① や ③ が 502 UPSTREAM_ERROR	Places API (New) / Routes API が無効、キーの API 制限、課金未設定	手順 3 の確認コマンドで、キー単体で呼べるか試す
② ③ ⑦ ⑧ ⑩ が 502	Gemini のキーやモデル名の誤り、モデルが構造化出力に未対応	手順 4 の 2 つのコマンドが通るか確かめる。⑧ は失敗しても `failed` を返すだけで、⑨ は使える
429 RATE_LIMITED	1 人 1 分 120 回の制限、または Gemini・Maps の回数制限	1 分ほど待つ（`Retry-After: 60`）。Gemini の無料枠なら有料枠に切り替える
④ だけが 409 になる	複合インデックスがまだ作成中か、未作成（Firestore の FAILED_PRECONDITION を、サーバーが 409 として返している）	手順 2 のインデックスが「有効」になるまで待つ
⑥⑪⑭⑮ が 409 VERSION_CONFLICT	`If-Match` の version が古い	⑤ で最新の version を